Homework-2

1) Define an integer array. Declare an integer pointer and initialize it to point to the first element of the array. Use pointer arithmetic to traverse the array and print the elements.

```
#include <stdio.h>

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    int i;

    printf("Array elements are:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d ", *(p + i));
    }

    return 0;
}
```

Output:

```
Array elements are:
10 20 30 40 50
```

2) Calculate String Length Using Pointer.

```
#include <stdio.h>

int main()
{
    char str[100];
    char *p;
    int length = 0;

    printf("Enter a string: ");
    scanf("%s", str);

    p = str;

    while(*p != '\0')
    {
        length++;
        p++;
    }

    printf("Length of string = %d", length);

    return 0;
}
```

Output:

```
Enter a string: Tamilnadu
Length of string = 9
```

3) Write a function on that calculate both the sum and product of two numbers. Return these two results using pointers parameters.

```
#include <stdio.h>

void calculate(int a, int b, int *sum, int *product)
{
    *sum = a + b;
    *product = a * b;
}

int main()
{
    int a, b, sum, product;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    calculate(a, b, &sum, &product);

    printf("Sum = %d\n", sum);
    printf("Product = %d\n", product);

    return 0;
}
```

Output:

```
Enter two numbers: 50 25
Sum = 75
Product = 1250
```

4) Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer to access, modify and iterate the records.

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
    int id;
    char name[30];
    float mark;
};

int main()
{
    int n, i;
```

```
    printf("Enter number of students: ");
    scanf("%d", &n);

    struct Student *s;

    s = (struct Student *)malloc(n * sizeof(struct Student));

    for(i = 0; i < n; i++)
    {
        printf("\nEnter details of student %d\n", i + 1);

        printf("ID: ");
        scanf("%d", &(s+i)->id);

        printf("Name: ");
        scanf("%s", (s+i)->name);

        printf("Mark: ");
        scanf("%f", &(s+i)->mark);
    }

    printf("\nStudent Details:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d %s %.2f\n",
               (s+i)->id,
               (s+i)->name,
               (s+i)->mark);
    }

    free(s);

    return 0;
}
```

Output:

```
Enter number of students: 2

Enter details of student 1
ID: 201
Name: charu
Mark: 98

Enter details of student 2
ID: 203
Name: hema
Mark: 92

Student Details:
201 charu 98
203 hema 92
```

5) Define two functions with same signature.Declare a function pointer and use the pointer to call both function.

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int (*fp)(int, int);

    fp = add;
    printf("Addition = %d\n", fp(5, 3));

    fp = multiply;
    printf("Multiplication = %d\n", fp(5, 3));

    return 0;
}
```

Output:

```
Addition = 8
Multiplication = 15
```

6) Declare an integer a pointer to it and a double pointer.print the values and addresses at each level.

```
#include <stdio.h>

int main()
{
    int x = 10;

    int *p = &x;
    int **q = &p;

    printf("Value of x = %d\n", x);
    printf("Address of x = %p\n", (void *)&x);

    printf("Value using p = %d\n", *p);
    printf("Address stored in p = %p\n", (void *)p);

    printf("Value using q = %d\n", **q);
    printf("Address stored in q = %p\n", (void *)q);
```

```
    return 0;
}
```

Output:

```
Value of x = 10
Address of x = 0x7ffeab527934
Value using p = 10
Address stored in p = 0x7ffeab527934
Value using q = 10
Address stored in q = 0x7ffeab527928
```

7) Sort an array of floating point numbers using selection sort.

```
#include <stdio.h>

int main()
{
    float a[5], temp;
    int i, j, min;

    printf("Enter 5 floating point numbers:\n");

    for(i = 0; i < 5; i++)
        scanf("%f", &a[i]);

    for(i = 0; i < 4; i++)
    {
        min = i;

        for(j = i + 1; j < 5; j++)
        {
            if(a[j] < a[min])
                min = j;
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Sorted array:\n");

    for(i = 0; i < 5; i++)
        printf("%.2f ", a[i]);

    return 0;
}
```

Output:

```
Enter 5 floating point numbers:
5.8 2.5 4.6 1.2 3.1
Sorted array:
```

1.20 2.50 3.10 4.60 5.80

8) Define a structure containing student ID,name and grade.create an array of student structures and sort them based on different fields.

```
#include <stdio.h>
#include <string.h>

struct Student
{
    int id;
    char name[30];
    float grade;
};

int main()
{
    struct Student s[3], temp;
    int i, j;

    printf("Enter details of 3 students:\n");

    for(i = 0; i < 5; i++)
    {
        printf("\nStudent %d\n", i + 1);

        printf("ID: ");
        scanf("%d", &s[i].id);

        printf("Name: ");
        scanf("%s", s[i].name);

        printf("Grade: ");
        scanf("%f", &s[i].grade);
    }

    for(i = 0; i < 4; i++)
    {
        for(j = i + 1; j < 5; j++)
        {
            if(s[i].id > s[j].id)
            {
                temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    printf("\nStudents sorted by ID:\n");

    for(i = 0; i < 5; i++)
    {
        printf("%d %s %.2f\n",
```

```
            s[i].id, s[i].name, s[i].grade);
    }

    return 0;
}
```

Output:

```
Enter details of 3 students:

Student 1
ID: 103
Name: Arun
Grade: 78

Student 2
ID: 101
Name: Ravi
Grade: 85

Student 3
ID: 105
Name: Kumar
Grade: 92

Students sorted by ID:
101 Ravi 85.00
103 Arun 78.00
105 Kumar 92.00
```

9) Sort a set of negative numbers using radix sort.

```
#include <stdio.h>
#include <stdlib.h>

void countingSort(int a[], int n, int exp)
{
    int output[n];
    int count[10] = {0};
    int i;

    for(i = 0; i < n; i++)
        count[(a[i] / exp) % 10]++;

    for(i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for(i = n - 1; i >= 0; i--)
    {
        output[count[(a[i] / exp) % 10] - 1] = a[i];
        count[(a[i] / exp) % 10]--;
    }

    for(i = 0; i < n; i++)
```

```
        a[i] = output[i];
}

void radixSort(int a[], int n)
{
    int i, max = a[0];

    for(i = 1; i < n; i++)
        if(a[i] > max)
            max = a[i];

    for(int exp = 1; max / exp > 0; exp *= 10)
        countingSort(a, n, exp);
}

int main()
{
    int a[] = {-45, -12, -89, -5, -30};
    int n = 5;

    printf("Original array:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");

    for(int i = 0; i < n; i++)
        a[i] = abs(a[i]);

    radixSort(a, n);

    for(int i = 0; i < n; i++)
        a[i] = -a[i];

    for(int i = 0; i < n / 2; i++)
    {
        int temp = a[i];
        a[i] = a[n - i - 1];
        a[n - i - 1] = temp;
    }

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Output:

```
Original array:
-45 -12 -89 -5 -30
```

```
Sorted array:
-89 -45 -30 -12 -5
```

10) Write a c program to copy one array to another using pointers.

```
#include <stdio.h>

int main()
{
    int a[5], b[5];
    int *p, *q;
    int i;

    printf("Enter 5 elements:\n");

    for(i = 0; i < 5; i++)
        scanf("%d", &a[i]);

    p = a;
    q = b;

    for(i = 0; i < 5; i++)
    {
        *(q + i) = *(p + i);
    }

    printf("Copied array:\n");

    for(i = 0; i < 5; i++)
        printf("%d ", *(q + i));

    return 0;
}
```

Output:

```
Enter 5 elements:
100 200 300 400 500
Copied array:
100 200 300 400 500
```